NeverStop | By Sourabh Gorai

Introduction to Java

ICSE Computer Science | Class 9

Unit 1 — Part 4 Study Notes

Types of Programs • Compilation Process • JVM

Beginner Friendly | **Step-by-Step** | **Exam Ready** | **With Examples**

What You Will Learn in This Document

01	What is Java? — History & Introduction
02	Types of Java Programs — Applets vs Applications
03	Java Compilation Process — Step by Step
04	Source Code, Byte Code & Object Code Explained
05	Java Virtual Machine (JVM) — The Heart of Java
06	Quick Revision & Key Terms

SECTION 1 | Introduction to Java

What is Java?

Java is one of the most popular and widely-used programming languages in the world. It was created by **James Gosling** and his team at **Sun Microsystems** in 1991 and officially launched in **1995**. Today, it is owned and maintained by **Oracle Corporation**. The language was originally called **Oak** — named after an oak tree outside Gosling's office window — but was later renamed **Java**, inspired by Java coffee from the Indonesian island of Java. ■

■ **Fun Fact**

Java runs on more than 3 billion devices worldwide — including your Android phone, ATM machines, Smart TVs, Blu-ray players, and even NASA satellites!

Official Definition

■ **Definition — Java**

Java is a high-level, object-oriented, platform-independent programming language designed to have as few implementation dependencies as possible. Its core philosophy is: "Write Once, Run Anywhere" (WORA).

Three Key Terms Explained

Term	Meaning	Simple Analogy
High-Level Language	Code is written in simple English-like instructions. You don't need to talk to the machine in 0s and 1s.	Like giving directions in plain English instead of GPS coordinates.
Object-Oriented	Programs are built using 'objects' — just like real-world things. A car object has colour, speed, and actions like start and stop.	Think of Lego bricks — each brick (object) has properties and fits with others.
Platform-Independent	A Java program written on Windows runs perfectly on Mac, Linux, or Android — without any changes.	Like a movie DVD that plays on any DVD player, regardless of brand.

■ Remember

"Write Once, Run Anywhere" (WORA) is Java's most famous principle. It means you write your Java program once and it can run on any device — Windows, Mac, Linux, Android — without rewriting a single line of code!

SECTION 2 | Types of Java Programs

Java programs are broadly classified into two types — **Applets** and **Applications**. Each has a different purpose, environment, and structure.

Feature	Java Applet	Java Application
Where it runs	Inside a web browser	Directly on the computer / JVM
Internet needed	Yes (mostly)	No
main() method	Not present	Must be present
Base class	Extends Applet class	No special base class needed
Status today	Deprecated (no longer used)	Widely used everywhere
Example	Old interactive web widgets	WhatsApp, Acrobat Reader, Android apps

2.1 Java Applets

■ Definition — Java Applet

A Java Applet is a small Java program that is designed to run inside a web browser. It is embedded in a webpage and downloaded over the internet to run on the user's machine. Applets extend the java.applet.Applet class and do not have a main() method.

Key Characteristics of Java Applets:

- **Runs inside a browser:** No separate installation needed by the user.
- **No main() method:** The browser controls the lifecycle using init(), start(), stop(), and destroy() methods.
- **Security restricted:** Applets cannot access your local files — this was a safety feature.
- **Requires Java Plugin:** The browser needed a Java plugin to run applets.

Example — A Simple Applet:

```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet {
public void paint(Graphics g) {
g.drawString("Hello, World!", 50, 50);
}
}
```

Notice: There is no main() method. The browser calls paint() automatically to display content.

■■ Important Note

Java Applets are now DEPRECATED. Modern browsers (Chrome, Firefox, Edge) no longer support them. However, they remain in the ICSE syllabus because understanding applets helps you grasp Java's history, browser interaction, and program lifecycle concepts.

2.2 Java Applications

■ Definition — Java Application

A Java Application is a standalone program that runs directly on the Java Virtual Machine (JVM) — without needing a web browser. It always has a main() method, which is the starting point of the program. Java applications can be console-based, GUI-based, or web-based.

Types of Java Applications:

Type	Description	Real-World Example
Console Application	Text-based program. Output is shown in the terminal/command prompt. Used for learning and logic practice.	A program that prints 'Hello World' or calculates your CGPA.
GUI Application (Desktop)	Has a graphical window with buttons, menus, and text boxes. Built using Java's Swing or JavaFX library.	Adobe Acrobat Reader — its windows, buttons, and scroll bars were built using Java GUI.
Web Application	Runs on a web server. Users access it via a browser. Uses Java technologies like Servlets and JSP.	SBI Net Banking, Flipkart, IRCTC ticket booking system.

Mobile Application	Apps built for Android devices. Android's original language was Java (now Kotlin, which is based on Java).	WhatsApp (original Android version), Gmail App, Google Maps.

Example — A Simple Console Application:

```
public class HelloWorld {
public static void main(String[] args) {
System.out.println("Hello, World!");
}
}
```

This is the simplest Java application. The main() method is where execution always begins.

SECTION 3 | The Java Compilation Process

When you write a Java program, your code goes through a fascinating journey before it actually runs on a computer. This journey involves three stages: writing **Source Code**, compiling it into **Byte Code**, and finally the JVM converting it to **Object Code**. Let's explore each stage in detail.

The Complete Compilation Journey:

Step 1 — Writing Source Code

■ Definition — Source Code

Source Code is the human-readable Java program written by the programmer. It is stored in a file with the .java extension. Source code uses Java syntax — keywords, variables, methods — that a human can read and understand.

Source code is what *you* write in a text editor or IDE like BlueJ, Eclipse, or VS Code. It is readable English-like code, but the computer cannot run it directly — it needs to be translated first.

Example Source Code (saved as MyProgram.java):

```
// This is a source code file – MyProgram.java
public class MyProgram {
public static void main(String[] args) {
System.out.println("Java is amazing!");
}
}
```

Step 2 — Compilation (Creating Byte Code)

Once you have written your source code, you run the **Java Compiler** using the command:

```
javac MyProgram.java
```

What does the compiler do?

- Reads your **.java** source file from top to bottom.

- Checks for **syntax errors** — spelling mistakes, missing semicolons, wrong structure.
- If no errors are found, it converts your source code into **Byte Code**.
- Creates a new file called **MyProgram.class** containing the byte code.

■ Definition — Byte Code

Byte Code is the intermediate output produced by the Java compiler. It is stored in a .class file. Byte code is not human-readable and cannot run directly on any operating system. It is specifically designed to be understood and executed by the Java Virtual Machine (JVM). This is what makes Java platform-independent!

■ Key Insight — Why Byte Code?

Traditional languages like C++ compile directly to machine code for one specific OS. Java compiles to byte code instead — a universal intermediate language that any JVM on any OS can execute. This is exactly why Java is platform-independent!

Step 3 — Execution by JVM (Object Code)

To run the compiled byte code, you use the **java** command:

```
java MyProgram
```

The JVM reads the **.class** byte code file and converts it into **Object Code** (also called Machine Code) — a series of 0s and 1s that your computer's processor can directly execute.

■ Definition — Object Code (Machine Code)

Object Code is the lowest-level code that a computer's CPU can directly understand and execute. It consists entirely of binary digits (0s and 1s). The JVM internally converts byte code into object code specific to the operating system it is running on.

C++ vs Java — How They Compile:

	C++ (Traditional)	Java
Compiler Output	Machine Code directly	Byte Code (.class file)
Platform-Specific	Yes — different build for each OS	No — one byte code for all OS
Who runs it	OS directly runs it	JVM runs it on any OS

Portability	Low — must recompile	High — Write Once, Run Anywhere

SECTION 4 | Java Virtual Machine (JVM)

What is the JVM?

■ Definition — Java Virtual Machine (JVM)

The Java Virtual Machine (JVM) is a software-based virtual machine that reads Java byte code (.class files) and converts it into machine code specific to the operating system it is running on. The JVM is what makes Java platform-independent — it acts as a bridge between the byte code and the actual hardware.

The word "Virtual Machine" might sound complex, but it simply means a **software-simulated computer**. The JVM is not a physical machine — it is a program that behaves like a computer and understands Java byte code. Every OS has its own version of the JVM, but they all understand the same byte code.

■ Perfect Analogy — The Universal Translator

Imagine a United Nations conference where delegates speak different languages. A translator listens to one common language (byte code) and translates it into each delegate's local language (Windows, Mac, Linux machine code). The JVM is exactly that translator for Java!

What Does the JVM Do? — Internal Components:

Component	Role	Simple Analogy
Class Loader	Loads the .class byte code files into memory when the program starts.	Like a librarian fetching the right book before you read it.
Bytecode Verifier	Checks that the byte code is safe and does not violate Java's security rules before running.	Like a security guard checking your bag before entry.
Interpreter	Reads and executes byte code line by line, converting each instruction to machine code.	Like a translator who translates a speech sentence by sentence.
JIT Compiler (Just-In-Time)	Compiles frequently-used byte code into machine code all at once, making the program run faster.	Like translating an entire document at once instead of word by word.

Garbage Collector	Automatically finds and frees memory that is no longer being used by the program.	Like a cleaner who automatically picks up trash so you don't have to.

JVM vs JRE vs JDK — What Is the Difference?

These three terms are often confused. Here is a clear explanation:

JDK (Java Development Kit)	JRE (Java Runtime Environment)	JVM (Java Virtual Machine)
The complete toolkit for developers. Contains: JRE + Compiler (javac) + Debugging Tools. Used to WRITE and RUN Java programs.	The environment needed to RUN Java programs. Contains: JVM + Standard Libraries. Used when you only want to run (not develop) Java apps.	The core engine that runs Java byte code. Converts byte code to machine code. Contained inside both JRE and JDK.
Think of it as: A fully equipped KITCHEN for a Chef (Developer)	Think of it as: A DINING ROOM — only for eating (running)	Think of it as: The STOVE — the actual cooking/running engine

■ Simple Rule

If you are a STUDENT or DEVELOPER writing Java code — install the JDK. It contains everything you need: compiler, JVM, and all libraries. The JDK download is available free from oracle.com.

SECTION 5 | Quick Revision & Key Terms

Key Terms — At a Glance

#	Term	One-Line Definition
	Java	High-level, OOP, platform-independent language. Created by James Gosling, 1995.
02	**Applet**	Java program that runs inside a web browser. Now deprecated.
	Application	Standalone Java program with a main() method. Used everywhere today.
04	**Source Code**	Human-readable Java code written by the programmer. Stored as .java file.
	Java Compiler	Tool (javac) that converts source code into byte code.
06	**Byte Code**	Intermediate code output by the compiler. Stored as .class file. Platform-neutral.
	Object Code	Binary machine code (0s and 1s) the CPU directly executes. Generated by JVM.
08	**JVM**	Java Virtual Machine — runs byte code on any platform by converting it to machine code.
	JRE	Java Runtime Environment — JVM + libraries. Used to run Java programs.
10	**JDK**	Java Development Kit — JRE + compiler + tools. Used to develop Java programs.
	WORA	"Write Once, Run Anywhere" — Java's platform-independence guarantee.
12	**main() method**	Entry point of every Java application. Execution always starts here.

The Complete Java Journey — Summary Diagram

Practice Questions — Check Your Understanding

Q1.	What does WORA stand for, and which feature of Java does it describe?
Q2.	What is the difference between a Java Applet and a Java Application?
Q3.	What is the file extension of: (a) Source Code (b) Byte Code?
Q4.	Why is Java called a platform-independent language?
Q5.	What is the role of the Java compiler? What command is used to compile a Java program?
Q6.	Define Byte Code. Why is it called an 'intermediate' language?
Q7.	What is the JVM? How is it different from the JRE and JDK?
Q8.	Why are Java Applets no longer used in modern browsers?
Q9.	What does the Garbage Collector in the JVM do?
Q10.	Write the simplest Java application that prints your name on the screen.

■ Coming Up in Video 2

In the next part, we will explore the powerful Features of Java — Simple, Robust, Secure, Object-Oriented, Platform-Independent, and many more — each explained with real-world examples. We will also look at the amazing real-time applications that use Java: Android apps, banking systems, gaming, robotics, chatbots, and virtual assistants!